

AIMS AI Pipelines

Two AI features. What each one does, step by step, and what stops it going wrong.

Services 2 Model openai/gpt-oss-120b Provider Groq Model calls per question 1
 Rows sent to the model 0

What AIMS has

Two separate AI features. They are kept apart on purpose, because they answer different questions and carry different risk.

ASK THE DATA Answers “what is the number” You type a question. It returns real rows from your database, as a table or a chart. /api/analytics	HELP CHATBOT Answers “how do I” You ask how AIMS works. It replies from written documentation. It never touches your records. /api/chatbot
---	--

Both call the same model through **Groq**.

Neither can write to the database. Not insert, not update, not delete.

The chatbot holds **no database tools at all**. There is nothing for a crafted question to talk it into reaching.

They were once one service. Splitting them fixed a real failure.

The rule everything is built around

Row data never enters the model. The model turns your question into a plan. The database answers the plan. The rows go to your browser without passing back through the model.
WHY THIS RULE EXISTS The old combined service fetched rows, pasted a sample into the prompt, and asked the model to describe them. It reported 200 fee defaulters, all with zero paid. The real figures: 1,175 defaulters, of whom 788 had paid something. The model was not at fault. Our own code applied <code>LIMIT 200</code> , counted the array <i>after</i> the limit, and wrote “the total is 200” into the prompt. The model obeyed. No model swap could have fixed that. Only removing the round trip could.

Now nothing is pasted into a prompt. A count on screen is the length of the array beside it.

The two, side by side

	Ask the Data	Help Chatbot
Answers	What the number is	How AIMS works
Route	/api/analytics	/api/chatbot
Reads from	The live database, read-only	33 documentation files
Who can use it	Super Admin, Admin, Teacher	Those three, plus Student and Parent
The model writes	A JSON plan you never see	The reply you read
Temperature	0 – a plan is not prose	0.3 – prose
Model sees your rows	Never	No rows exist here
Conversation history	None, by design	Last 4 turns
Database access	Read-only account	None at all
Needs Docker	No	Yes – Qdrant
Worst case	A wrong number – why it is built this way	A clumsy explanation

Students are deliberately excluded from Ask the Data. Every figure a student may see already has its own screen. A free-form query channel over one record buys nothing but risk.

Ask the Data — step by step

Deterministic code – same input, same output		The model decides something	
1	Question arrives	CODE	Role checked. Rate limit checked. Up to 500 characters – longer is usually a paste.
2	Catalogue is built	CODE	Only the tools this role may call, plus a schema trimmed to the tables the question mentions. An admin sees 28 tools; a teacher sees 18.
3	Model writes a plan	MODEL	It sees the question and the catalogue. It sees no rows . It returns strict JSON: which tool, which arguments, which chart, a title. Temperature 0 .
4	Plan is validated	CODE	Model output is untrusted text. A plan naming a tool outside this role’s catalogue is refused and written to the audit log – even though nothing ran.
5	Database runs it	CODE	Through the read-only account. A teacher’s query is rewritten to their own roster first. Cut off at 10 seconds.
6	Rows go to the browser	CODE	Untouched. Drawn by one of seven fixed templates – six charts and a plain table. The model never writes chart code and never picks a colour.

One step out of six uses the model. The rest is ordinary code you can read and test.

The model decides four things

- The typo-corrected version of your question.
- Which tool to call — or the SQL to write when no tool fits.
- Which chart shape to use.
- The title above it.

It never decides a number, a name, a count, a total, or who may see what.

The guards, in the order they run

#	Guard	What it stops
1	Tool registry	A role is never shown a tool it may not call. The name is absent from the prompt.
2	SQL guard	One statement only. Must start <code>SELECT</code> or <code>WITH</code> . Under 5,000 characters. 20 forbidden constructs. A <code>LIMIT</code> is forced on.
3	Scope rewrite	For teachers: 12 table names are redefined as CTEs over their own roster.
4	Read-only account	Holds <code>SELECT</code> and nothing else, with column-level limits. This is the real control.
5	Timeout	A runaway join is cut off at 10 seconds.

Layer 4 holds even if layers 1–3 are wrong.

A guard written in regular expressions is not a SQL parser. Layers 1–3 catch honest mistakes early and produce a readable refusal. The database account is what actually stands between the model and your data.

How a teacher is limited to their own students

Not done by appending `AND student_id IN (...)` to whatever the model wrote. That must be applied correctly to every join, subquery and union, every time. One miss leaks another teacher’s students.

Done instead by redefining the table names: `WITH students AS (SELECT ... WHERE student_id IN (their roster))` , and eleven more.

MySQL resolves a CTE ahead of a real table. **The model cannot forget the filter, because the model never writes the filter.**

A teacher who teaches nobody gets `IN (NULL)` everywhere and sees nothing. That is correct, not an edge case.

Two ways to answer, one way to decline

Mode	When	What runs
tool	A ready-made tool covers the question	Hand-written SQL, reviewed and audited. Preferred.
sql	No tool fits	SQL the model wrote, then guarded.
refuse	Not a database question, or asks to change data	Nothing. Ask the Data is read-only.

Tools are plain functions with a fixed SQL statement inside. The model chooses which to call and supplies arguments — never the query itself. The menu puts the pre-built `vw_*` views first, because each already

knows what a term like “defaulter” means, so the model does not have to reconstruct that definition and get it subtly wrong.

What the read-only account refuses

Attempt	Result
SELECT password_hash FROM users	blocked
SELECT cnic_bform FROM students	blocked
SELECT salary FROM employees	blocked
SELECT * FROM payroll	blocked
SELECT * FROM assistant_messages	blocked
INSERT INTO programs ...	blocked

The transcript tables being unreadable matters more than it looks: a generated query cannot read other people’s conversations.

Presentation mistakes are corrected. Access mistakes are refused.

A plan asking for an impossible chart quietly becomes a table. A plan asking for data you may not see is rejected outright. Never the other way round.

Help Chatbot — step by step

1	Question arrives <code>CODE</code> Role checked. Up to 1,000 characters. This service serves students and parents too.
2	Intent is checked <code>CODE</code> “What can you do” and off-topic questions are answered here, without searching anything.
3	Library is searched <code>CODE</code> The documentation, split at its own headings and stored in Qdrant. Your role filters the results before scoring, so a student never retrieves a staff passage in the first place.
4	Results are re-ranked <code>CODE</code> A small <code>+0.05</code> bonus for a passage written for your own role. It breaks near-ties; it cannot promote a weak passage.
5	One model call <code>MODEL</code> The surviving passages, the screen list for your portal, and the last 4 turns go into one prompt. The model writes the reply, with a citation for each passage used.

The model has no search tool.

Searching happens in code, before the model is called. So it can never skip the lookup and answer from memory — which would mean inventing a fee deadline that reads exactly like a real one.

What the model is never given

A search tool, or any choice about whether to look something up.

A database connection.

A single student record.

Two kinds of empty, told apart

Nothing scored high enough — a real gap in the documentation. The model must say so and name who to ask.

Qdrant is unreachable — a system fault. The model must say it cannot check right now.

Keeping these separate matters: **an outage must not read to the user as “undocumented”**.

Screen names are not searched

Portal navigation is a plain list in code, not in Qdrant.

A route is a small exact fact. Putting it through a similarity search risks confidently sending someone to a screen that does not exist.

Hardcoded, it is either right or obviously wrong when a route moves.

It is also why navigation questions kept working during a retrieval outage while policy questions did not.

Every setting, and what it means

ASK THE DATA		
Setting	Value	Why
Model	openai/gpt-oss-120b	Reliably emits strict JSON, which the planner depends on.
Temperature	0	The same question should produce the same plan.
Max tokens	1200	Enough for a plan. The model does not write the answer.
Reasoning effort	low	Left at default it burned 406 of a 700-token budget and truncated the JSON. At low it uses 11. Picking a tool is classification, not proof.
Timeout	30s	Retried up to 3 times.
Row ceiling	50,000	Browser memory. Stated plainly if hit, never silently trimmed.
Rows drawn at once	100	DOM size. The rest are paged.
Max chart points	300	Above this it becomes a table, with the reason shown.
Question length	500 chars	Longer is usually a paste.
Query timeout	10s	A cartesian join is a plausible model mistake.
Plan cache	6 hours	A repeated question costs no tokens at all.
Rows shown to the model	0	Correctness. This is the one that mattered.

HELP CHATBOT		
Setting	Value	Why
Model	openai/gpt-oss-120b	Same model, different job.
Temperature	0.3	Explaining a policy reads badly at zero.
Max tokens	1024	Two or three sentences, sometimes a short list.

Passages kept	5	Effective value — see the note below.
Score floor	0.35	Effective value — see the note below.
Affinity bonus	+0.05	Breaks near-ties toward your own role. Cosine similarity has no idea who is asking.
History turns	4	The largest single line in the prompt, and the only one that grows.
Message length	1,000 chars	A support question is short.
Embedding model	all-MiniLM-L6-v2	384 dimensions, about 90 MB, runs on the server’s CPU. No API call, no cost.
Vector store	Qdrant	One collection, cosine distance. Runs in Docker.

KNOWN BUG – THE RETRIEVAL SETTINGS DO NOT TAKE EFFECT

`orchestrator.js` asks for `limit: topK × 3` candidates, so the affinity bonus has near-ties to reorder.

But `vectorStore.search()` reads the option as `topK`, not `limit`. So the request falls through to the defaults in `config/assistant.js`.

The effect: `config/chatbot.js` says top 10 at floor 0.30. What actually runs is **top 5 at floor 0.35**. Changing `CHATBOT_TOP_K` or `CHATBOT_MIN_SCORE` does nothing at all.

A one-word fix. Do it before tuning either number, because the current values were chosen against behaviour the code does not actually have.

SHARED LIMITS – PER ACCOUNT, NOT PER IP		
Window	Normal	Admin
Per minute	8	24
Per hour	60	180
Per day	200	600

Per account, because a campus shares IP addresses and an IP limit would throttle a whole computer lab as one user. These are budget protection, not abuse prevention.

When something breaks

Each failure costs one thing, not the whole system.

What fails	What happens
Docker is not running	Qdrant vanishes. Policy answers say the index is offline. Navigation still works. Ask the Data is unaffected.
A Groq key is rate-limited	The next key is used; the tired one rests as long as Groq asks. A rest under 20 seconds is waited out rather than failed.
Every Groq key is resting	503 with a real <code>Retry-After</code> .
Model returns broken JSON	Fences stripped, first object extracted. Then 422 asking you to rephrase.
Query names a column that does not exist	One repair call, given the real column list. Kept only if it works.
Database refuses a column	Reported as a refusal, never retried . That is the grants working.

MySQL is slow or down	The chatbot still answers. <code>saved:false</code> tells the UI not to promise a history entry.
Nothing scores high enough	Says it does not know, and names who to ask.
Chart cannot be drawn	Falls back to a table, with the reason shown. The rows are still correct.
Row ceiling reached	A separate <code>COUNT</code> finds the true total. The UI says “first N of M”.

THE FAILURE SHAPE WORTH WATCHING FOR

Retrieval once read `hit.payload.text`, but the field is called `content`. Every passage became an empty string and was dropped by the “no empty passages” filter.

Qdrant was up. Scores were healthy, 0.44 to 0.64. The chatbot still said nothing was documented — **indistinguishable from the index being down**.

The same shape appears in Ask the Data: a valid query that matches nothing reads as a fact about the institute rather than a vocabulary mismatch. **A correct-looking operation returning nothing, presented as an answer**. Watch for it anywhere a filter silently discards malformed input.

Where the model is used — and where it was removed

Job	Model?	Reasoning
Fix typos in the question	Yes	Open vocabulary. No rule covers it.
Pick which tool to run	Yes	A keyword router was tried and mis-picked.
Write SQL when no tool fits	Yes	Genuine judgement.
Choose a chart shape	Yes	An intent judgement — but only from seven names, and code overrules a bad pick.
Explain a policy from passages	Yes	Turning evidence into a sentence is what a model is for.
Count or summarise rows	Removed	This is what reported 200 instead of 1,175. It is <code>array.length</code> .
Decide whether to search	Removed	It sometimes chose not to, and answered from memory.
Choose chart columns	Removed	Derived from the real values instead.
Write chart code	Removed	Seven fixed templates.
Format a route list	Removed	A fixed component draws matched screens.
Decide who sees what	Never	Access is not a model decision.

Not overused.

Every remaining call does something genuinely linguistic — turning human wording into structured intent, or turning evidence into a sentence. Everything countable, renderable or checkable is in code.

The jobs marked **removed** are exactly the ones that produced the original wrong answer.

EVERY ANSWER IS TRACEABLE

Each response carries the tool name or the exact SQL, the timing, the planner token count, and `repaired: true` if the first query failed.

The canvas shows all of it under “How this was answered”, and every call is written to the audit log — so a disputed figure can be checked without asking an engineer.

What still needs work

Bugs and fixes

- Fix the retrieval option name.** `limit` versus `topK` in `vectorStore.search()` . Until then the chatbot’s tuning settings do nothing. Do this first.
- Add a retrieval smoke test.** The empty-passage bug passed every health check: Qdrant up, scores healthy, answers empty. A test asserting one known question returns a non-empty passage would have caught it.
- Watch `repaired: true` .** If it appears often, the schema digest needs widening — not the retry doing more work.

Handover risks

- Qdrant runs on one machine.** There is no hosted vector store. If that machine is off, the chatbot has no library.
- The Groq keys are personal.** They must be replaced with keys the organisation owns.
- Re-ingestion is manual.** Edit a documentation file and nothing changes until the ingest script is run again.
- The plan cache is in memory.** A restart empties it.

Improvements

- The tool catalogue is now the main cost** — a large share of every planner prompt. Narrowing to a few candidate tools first would roughly halve it.
- Value hints cover only 4 columns** — sections, programmes, departments, batches. Statuses and semester labels are the likely next gap, and they fail silently: a valid query matching nothing.
- Charts cap at 300 points.** A larger result becomes a table. An optional roll-up would make big results plottable.
- Grants are incomplete.** Some legitimate questions are refused at the database level. Whether to widen them is a decision, not a bug.
- There is no test set.** Answer quality is judged by reading answers, not measuring them.

CHECKED, NOT ASSUMED

Every setting on this page was read from the config files that ship with the code, not from memory.

The retrieval bug was confirmed by reading both sides:
`orchestrator.js` passes `limit` , `vectorStore.js` reads `topK` .

The 1,175 / 788 defaulter figures were checked against the live database.
The original combined assistant got all of them wrong.

AIMS AI Pipelines— how the two AI services work.

backend/src/services/{analytics,chatbot} · backend/src/services/assistant/{rag,tools} · frontend/src/pages/admin/AIAnalytics.jsx